

Hands on Lab 1
Single host profiling

SC 26 | Taming AI Storage Systems

DNN Checkpoint Save

When training a neural network, the training process periodically saves the network state and training parameters to storage.

We asked an LLM to code in C++

1. Create a fully connected DNN (Deep Neural Network) data structure.
2. Initialize the neuron's weights and biases using random numbers.
3. Store the DNN data structure to disk using Linux IO functions (open, write, close).

The created program imitates a **DNN checkpoint-save** workload.

NVIDIA

LLM Created Code

```
struct Neuron
{
   std::vector<double> Weights;
   double Bias = 0.0;
};

struct Layer
{
   std::vector<Neuron> Neurons;
};

struct DNN
{
   std::vector<Layer> Layers;
};
```

NVIDIA

LLM Created Code

```
void saveNeuralNetworkToFile(const DNN& network, const std::string& filename)
{
   int fd = open(filename.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0644);

   // Save the network's sizes
   uint64_t layersCount = network.Layers.size();
   uint64_t neuronsPerLayer = network.Layers[0].Neurons.size();
   ssize_t bytesWritten1 = write(fd, reinterpret_cast<const char*>(&layersCount), sizeof(layersCount));
   ssize_t bytesWritten2 = write(fd, reinterpret_cast<const char*>(&neuronsPerLayer), sizeof(neuronsPerLayer));

   // Output the neurons
   uint32_t layerIndex = 0;
   for (const Layer& layer : network.Layers)
   {
       for (const Neuron& neuron : layer.Neurons)
       {
           for (double weight : neuron.Weights)
           {
               ssize_t bytesWritten3 = write(fd, reinterpret_cast<const char*>(&weight), sizeof(weight));
           }

           ssize_t bytesWritten4 = write(fd, reinterpret_cast<const char*>(&neuron.Bias), sizeof(neuron.Bias));
       }
   }

   close(fd);
}
```

NVIDIA

Workload Performance

Running the generated code on a neural network having
- 100 Layers
- 1,000 Neurons in each layer

We get:

```
Creating new neural network
Operation time: 1.421 seconds

Saving neural network to /data/CheckPoint1.bin
Operation time: 61.83 seconds
```

NVIDIA

Instrument the code using NVTX

NVTX (NVIDIA Tools Extension), is a header only library, that annotates source code.
It is available at https://github.com/nvidia/nvtx
Tools can read the annotation to correlate profiling data to workload actions.

We added the following:

```
#include <nvtx3/nvtx3.hpp>
```

And in the relevant code place:

```
std::string layerTag = "Saving layer " + std::to_string(layerIndex++);
nvtx3::scoped_range layerRange(layerTag.c_str());
```

NVIDIA

Profiling the Workload

We profiled the program using Nsight Systems, a free performance analysis tool:

```
nsys profile -o ./CheckPointSave1.nsys-rep -y 40 -d 30 --storage-metric
--nfs-volumes=/home --trace='osrt,nvtx' --nic-metrics=lf ./CheckpointSave -w
/data/CheckPoint1.bin -l 100 -n 1000 -m posix
```

nsys - the Nsight Systems CLI
-o - output profile report file name
-y - start collecting data after 40 seconds
-d - set maximal profiling duration to 30 seconds
--storage-metrics - load the storage metrics plugin
--nfs-volumes - collect performance metrics from the “/home” NFS volume
--trace - libraries to be traced: Operating System Runtime libraries (POSIX functions), NVIDIA Tools Extension
--nic-metrics - collect NIC metrics at low frequency

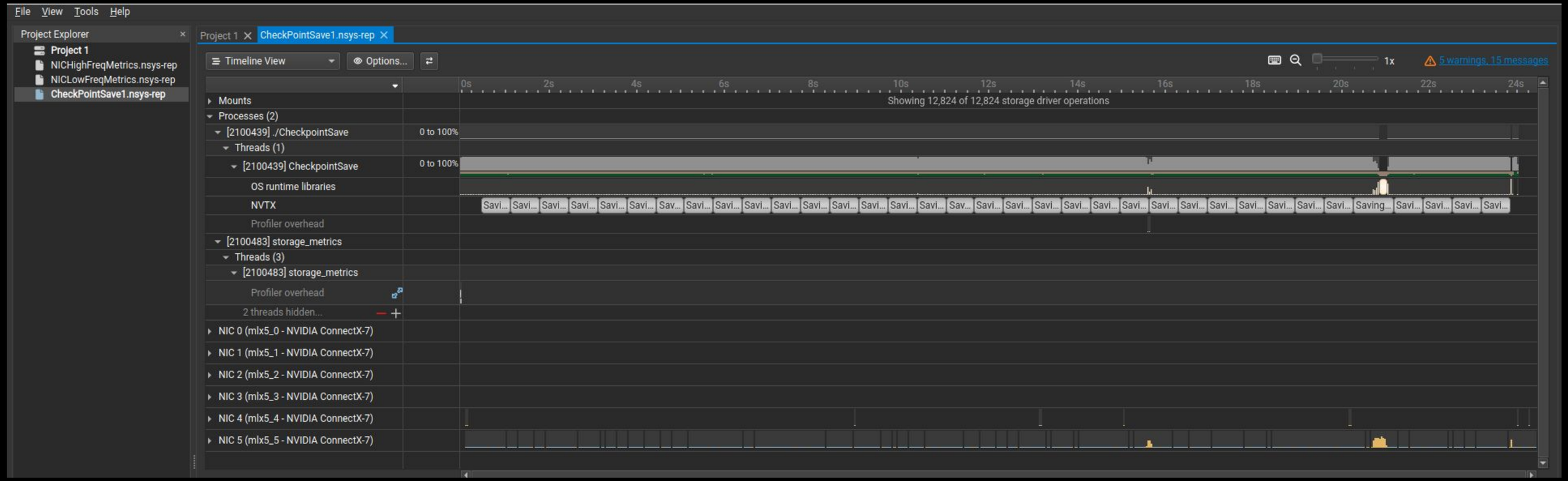
Profile Result

Please load the HandsOnLab1/ProfilerReportFiles/CheckPointSave1.nsys-rep profile report file into the Nsight Systems GUI:

1. nsys-ui
2. File -> Open
3. Choose HandsOnLab1/ProfilerReportFiles/CheckPointSave1.nsys-rep

NVIDIA

Profile Result



- Expand the different rows
- Zoom in/out using Ctrl + middle mouse wheel

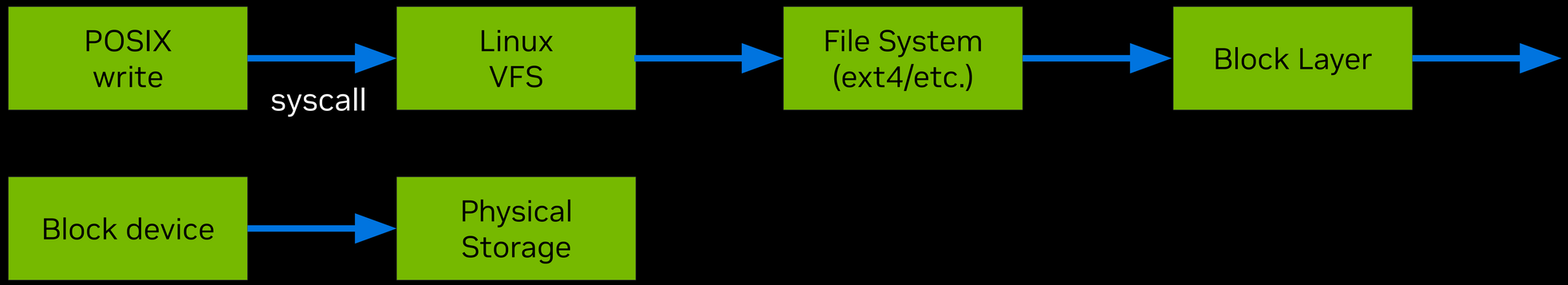
Try to figure out the displayed data

Profile Result Analysis

Each neuron’s weight is saved separately into the output file.

This created a huge amount of tiny write operations.

Each write operation triggers the following:



* For remote storage add (NIC -> Switch -> NIC)

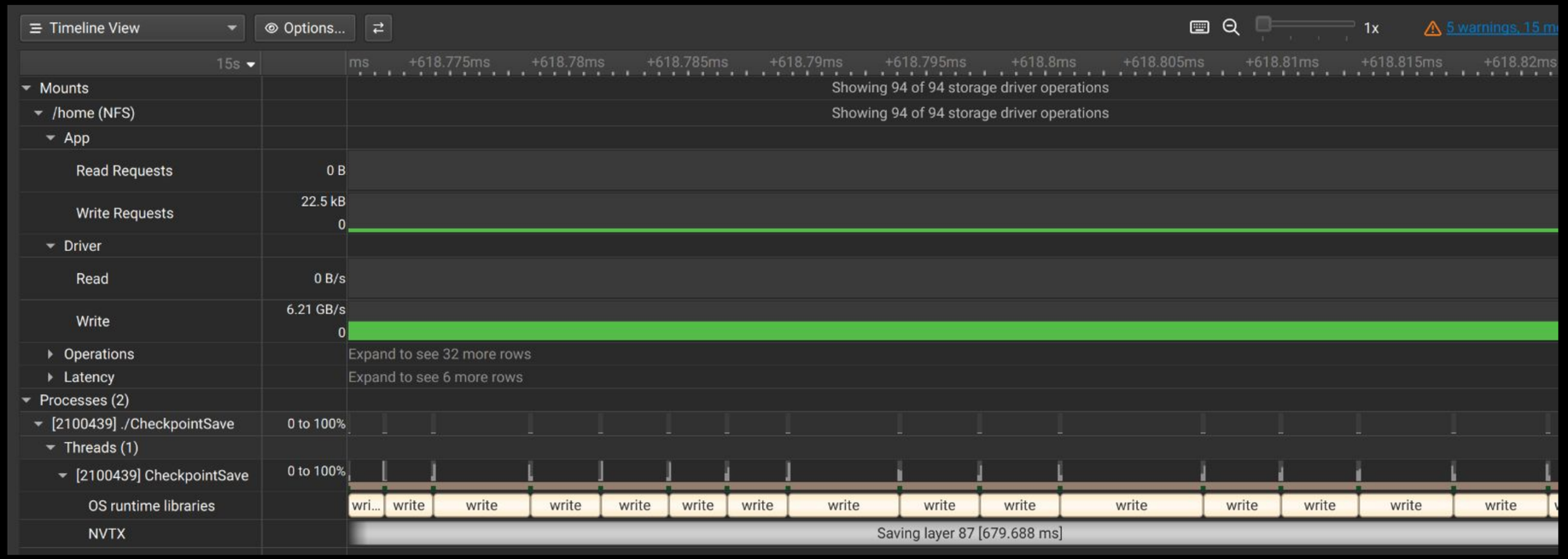
* For parallel FS add (Meta data server -> Object Storage Server)

NVIDIA

Profile Result Analysis

We see a huge amount of tiny I/O operations.

The file system layer buffers these writes to be more efficient, resulting in a fewer driver write requests.



Workload Improvement

To improve performance, we asked the LLM to:

- Store all layer's neurons into a buffer
- Write the layer's data into the file in a single write call.

From 100,100,000 write calls down to 100 write calls.

The Result is:

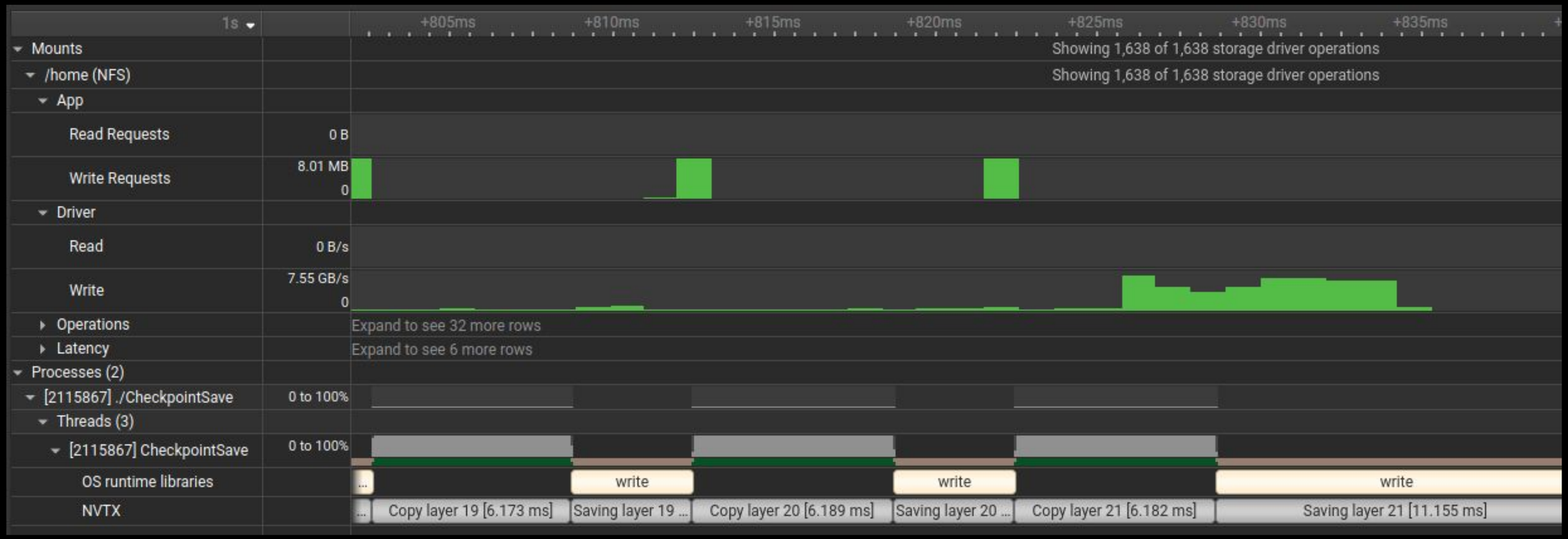
```
Saving neural network to /home/user/temp/CheckPoint2.bin
Save mode: posix_large_chunks
Operation time: 1.204 seconds
```

(Down from 61.83 seconds)

NVIDIA

Profile the Improved Workload

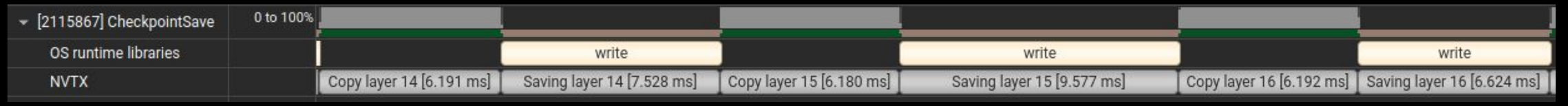
Please load the HandsOnLab1/ProfilerReportFiles/CheckPointSave2.nsys-rep profiler report file into the Nsight Systems GUI.



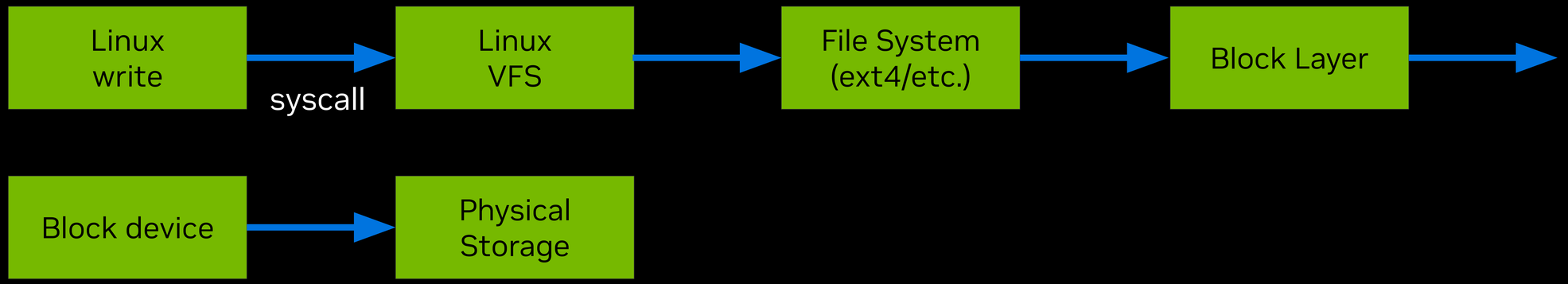
NVIDIA

Profile Result Analysis

Each layer (1,000 neuron) is saved into a buffer (“Copy Layer” NVTX range)
and then saved, using a single write operation, into the output file (“Saving Layer” NVTX range)



Remember the write flow:



NVIDIA

I/O Programming Best Practices

- Use large I/O operations (large buffers, group data, etc.)
- Binary formats are more efficient than textual formats
- Do concurrent compute and I/O (prefetch data before it is needed by compute)
- Consider using data compression
- Use multithreaded IO (carefully!)
- Use O_DIRECT to bypass caching system (when applicable)

```
int fd = open(filename.c_str(), O_WRONLY | O_CREAT | O_TRUNC | O_DIRECT);
```

- Use GPU Direct Storage to load data to/from the GPU
- Use optimized network and storage access libraries (don’t reinvent the wheel)

  These libraries have knobs to turn on features (GPU Direct Storage, Multithreading, etc.)
- Use tools to profile your workload

NVIDIA

Storage Optimized Libraries

- tf.data - The TensorFlow data loader (prefetch, map, cache, etc.)
- torch.utils.data.DataLoader - The PyTorch data loader (num_workers, pin_memory, non_blocking, etc.)
- NVIDIA Data Loading Library (DALI)
- Hugging Face datasets - Apache Arrow maps datasets directly from disk files into memory buffers
- HDF5 file format and library
- And many other